

INTRO TO DATA PIPELINES

Scenario

The fictional company **GameAnalytics** collects **daily gameplay event logs** (CSV format) from PlayStation consoles.

The logs are **millions of rows** per file and can easily exceed memory on a normal laptop.

Your job is to build a pipeline to:

1. **Extract** raw event files (provided as multiple CSVs, ~1–2 GB total).
 2. **Transform** the data:
 - Filter to only PS5 games
 - Parse timestamps to proper UTC datetime
 - Aggregate hourly concurrent players and total play-time per game
 - Detect malformed records and log them separately
 3. **Load** the cleaned/aggregated data into a **PostgreSQL** database for analysis.
-

Requirements & Hints

Research & Design

- Write a short design doc (one page max) covering:
 - Differences between **ETL**, **ELT**, **EL**
 - Which strategy you chose **and why** for this scenario
 - High-level architecture diagram of your pipeline

Data & Tools

- **Dataset:** Use any large, free dataset of your choice (I have attached the python script for you which you can use to create your dataset).
- **Database:** PostgreSQL (local).
- **Python Libraries** (all free):
 - `psycopg2` or `SQLAlchemy` for database writes
 - **Choose a library that can handle big files** (examples:
 - `polars` (Rust backend, blazing fast)
 - `dask.dataframe` (parallel, out-of-core)
 - or `pyarrow` for chunked parquet/CSV reading)
 - `logging` for pipeline logs

Note: They must **not** rely solely on pandas; it will fail for very large files.

Pipeline Steps

Design the pipeline as **discrete tasks** (functions or classes) such as:

1. `extract_files` – read raw CSVs in chunks/out-of-core
2. `clean_transform` – apply filtering, parsing, aggregations
3. `validate_data` – check for duplicates/nulls and log anomalies
4. `load_to_db` – write results into PostgreSQL
5. (Optional) `archive_raw` – move processed files to an archive folder

Each task should:

- log start & end times
- raise an exception on failure

Pipeline Runner

Create a **main runner** (e.g., `main_pipeline.py`) that:

- Executes tasks in sequence (like Airflow DAG)
- Catches exceptions:
 - logs the exact **step name** and **error message**
 - stops further steps if a failure occurs
- Writes a run summary to a log file (`pipeline_run.log`) with:
 - start/end time
 - status of each step (SUCCESS / FAILED)

Deliverables

Students must submit:

1. **Design Document** (Markdown or PDF)
 - ETL vs ELT vs EL explanation
 - Their chosen approach and reasoning
 - Architecture diagram
2. **Code Folder** with:
 - All Python scripts (pipeline steps + runner)
 - `requirements.txt`
3. **SQL Schema** used for loading results
4. **Sample Log Output** demonstrating a successful run and a simulated failure